Java Programming Master Map

Synthesizing Syllabus Theory and Lab Implementations

Theory Concept Card

JVM Mechanics & Execution



Theory Concept Card

OOP Foundations



JAVA

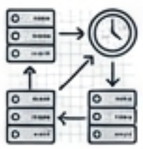
Theory Concept Card

Data Utilities & Structures



Theory Concept Card

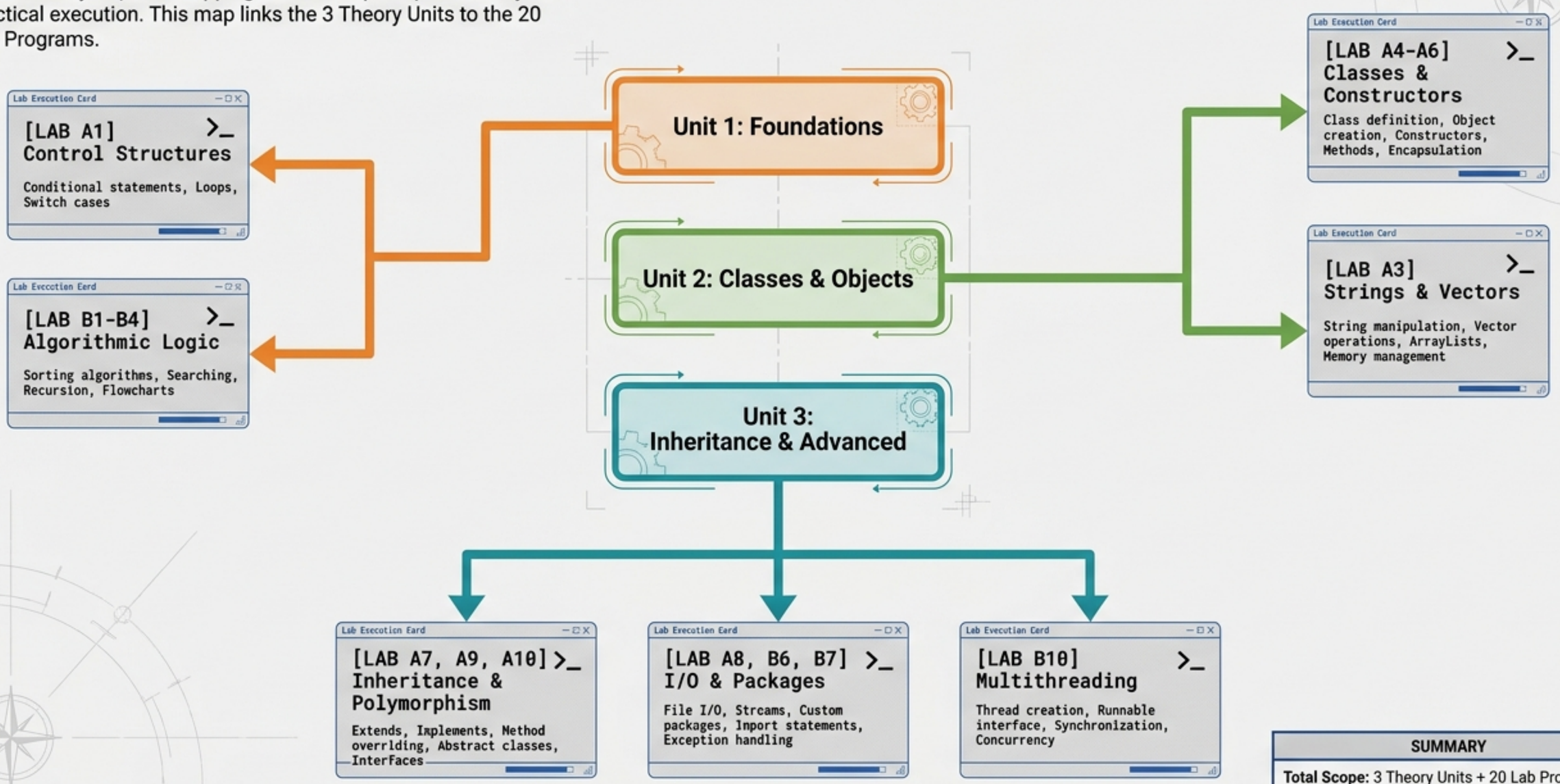
Concurrency & Enterprise



Gemini Notebook

The Master Exam Blueprint

Java mastery requires mapping theoretical principles directly to practical execution. This map links the 3 Theory Units to the 20 Lab Programs.



SUMMARY

Total Scope: 3 Theory Units + 20 Lab Programs.
Status: Ready for compilation.

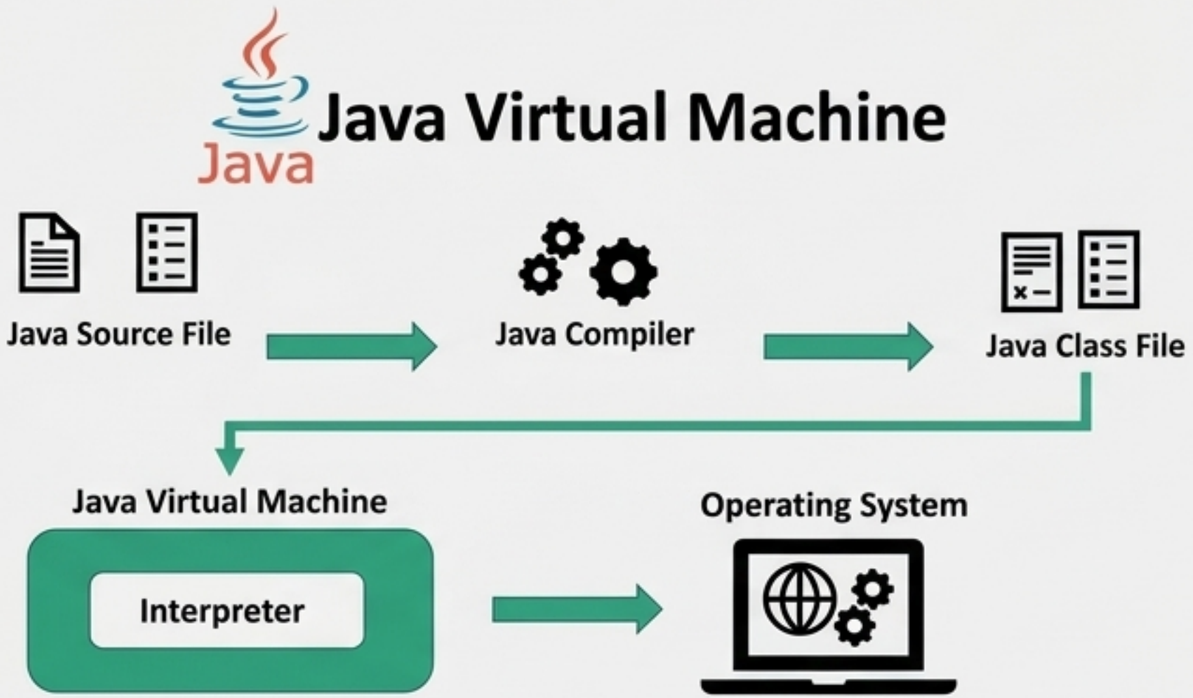
Gemini Notebook

The Execution Engine: JVM Architecture

Theory Concept Card

Write Once, Run Anywhere

The JVM converts platform-independent bytecode into machine-specific instructions.



Lab Execution Card

javac

The Java Compiler.
Converts .java into .class

Lab Execution Card

java

The Application Launcher.
Boots the JVM.

Lab Execution Card

javap

The Dis-assembler.
Inspects bytecode.

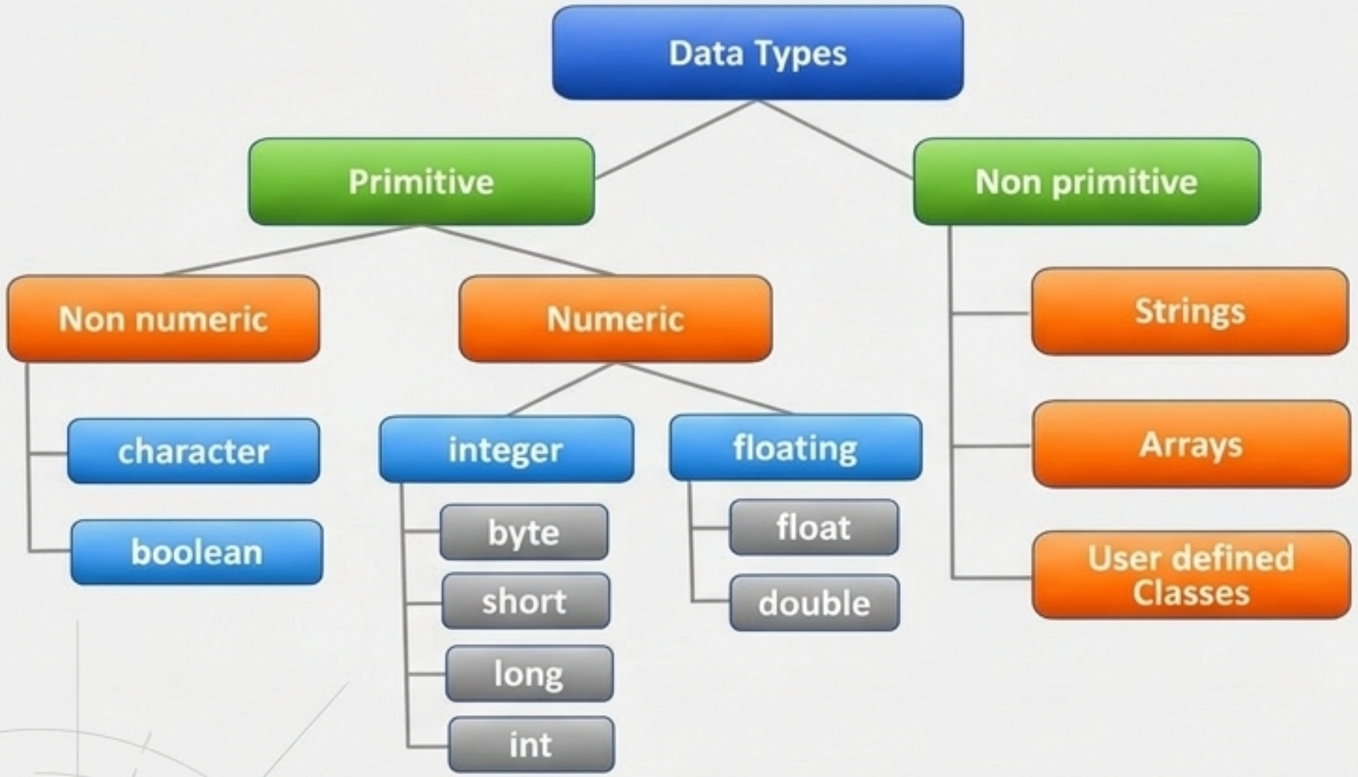
Lab Execution Card

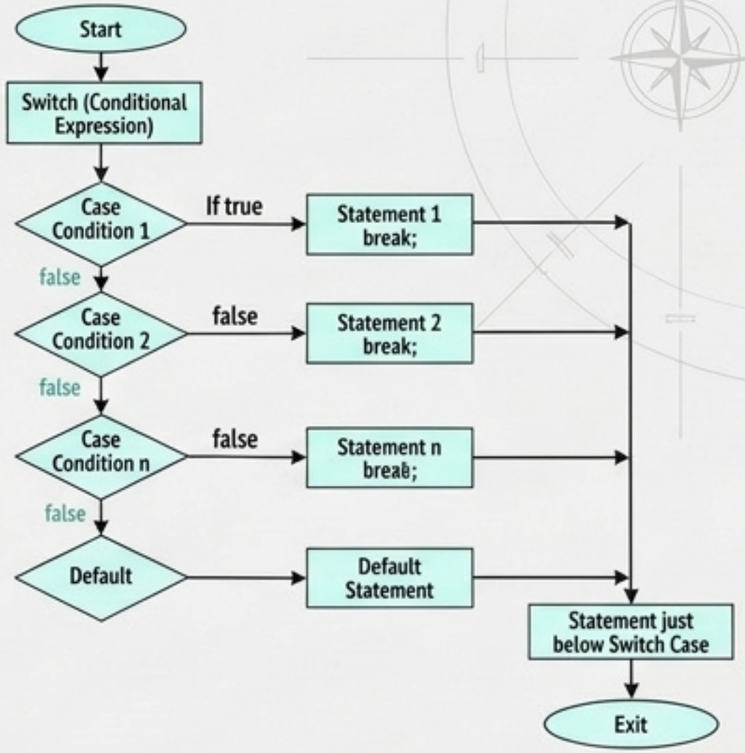
jdb

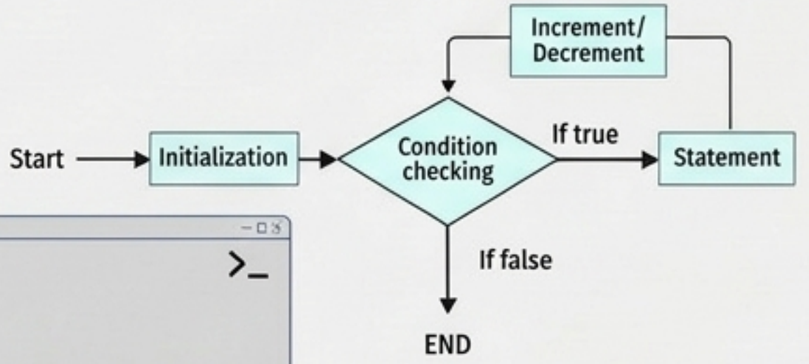
The Debugger.
Steps through code.

Gemini Notebook

Foundations:
Data Types & Control Flow







Lab Execution Card

```
[LAB INTEGRATION]
[LAB A1] Simple Calculator (switch / while)
[LAB B1] Prime Numbers (for loop)
[LAB B2] Palindrome Checker
[LAB B3, B4] Arrays: Name Sorting & Matrix Multiplication
```

Gemini Notebook

The OOP Blueprint: Classes & Constructors

Theory Cencept Card

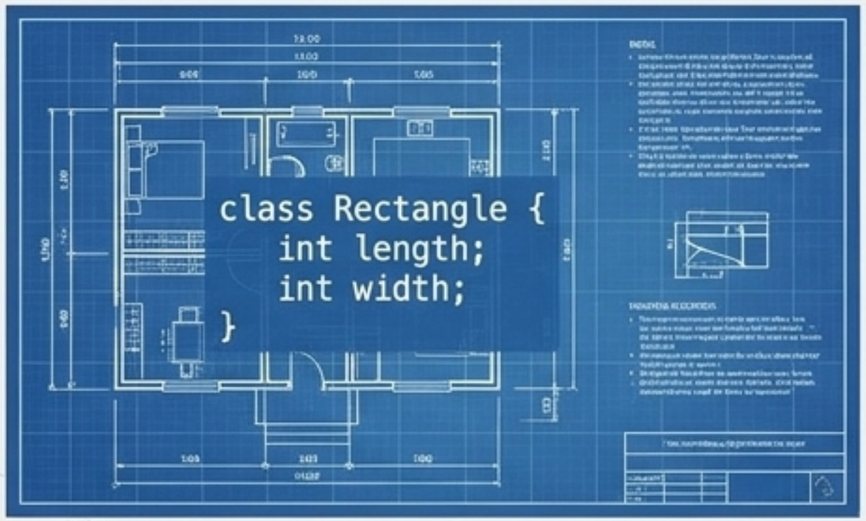
Class

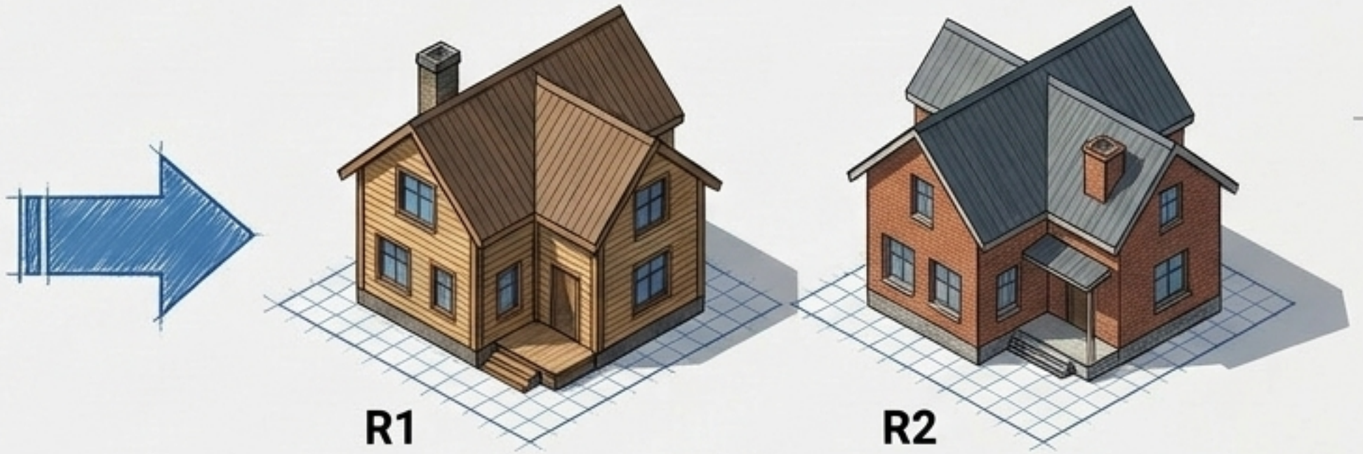
A user-defined template or blueprint with attributes (data) and methods (behaviors).

Theory Concept Card

Object

A real-world instance allocated in memory using the 'new' keyword.





Lse Erccotian Card

```
[LAB A5] Creation of classes and functions.
[LAB A4] Constructors.
 Types:  1. Default (no parameters)
         2. Parameterized (custom values)
         3. Copy (clones an object)
```

Gemini Notebook

The Four Pillars of OOP

Theory Concept Card



Encapsulation

Bundles data & methods into a single unit. Data is hidden via private modifiers.

Analogy: A capsule pill hiding its chemical mix.

Theory Concept Card

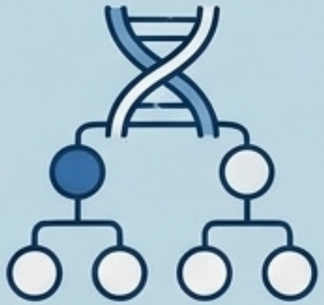


Abstraction

Hides complex implementation details, exposing only essential features.

Analogy: Pressing the gas pedal without understanding engine combustion.

Theory Concept Card



Inheritance

Child classes derive properties from parent classes to reuse code.

Analogy: Car and Truck inheriting from Vehicle.

Theory Concept Card



Polymorphism

One interface, many forms. Same action name does different jobs depending on the object.

Analogy: A universal 'Play' button streams music for audio, or video for movies.

Gemini Notebook

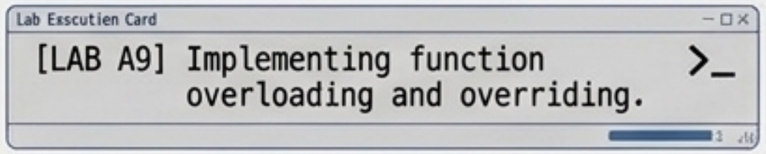
Diagnostic Matrix: Polymorphism & Visibility

Polymorphism

Method Overloading (Compile-Time)	Method Overriding (Run-Time)
Same method name, different parameters within the same class.	Subclass provides a specific implementation of a method already defined in its parent class.

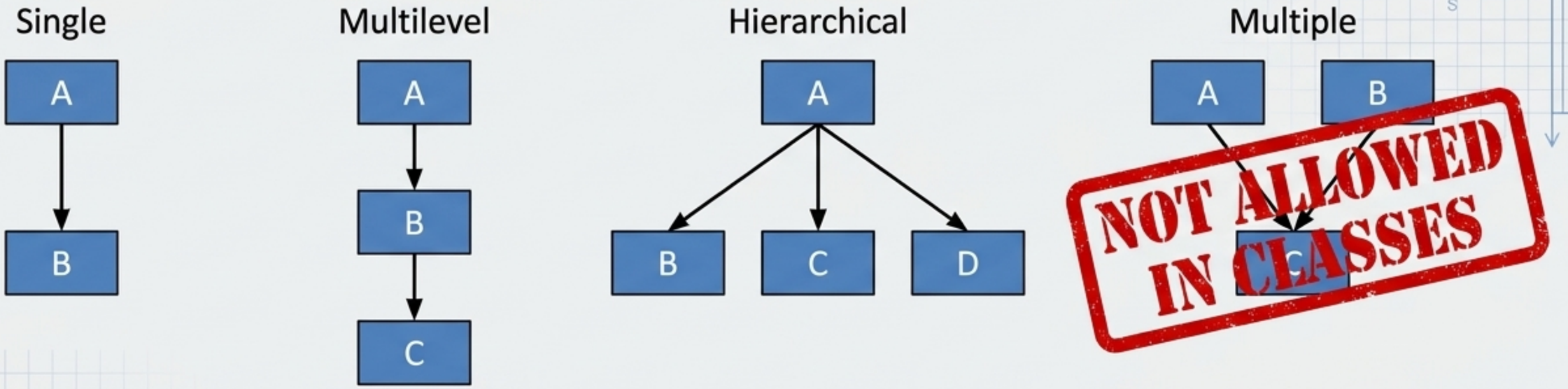
Visibility Matrix

Modifier	Same Class	Same Package	Subclass	Outside Package
`public`	✅	✅	✅	✅
`protected`	✅	✅	✅	❌
`default`	✅	✅	❌	❌
`private`	✅	❌		



Gemini Notebook

Inheritance Hierarchies & Interfaces



Theory Concept Card

The Resolution: Interfaces

Java strictly forbids Multiple Inheritance in classes to avoid ambiguity. The fix is Interfaces. Interfaces contain only final fields and abstract methods, allowing a class to implement multiple interfaces simultaneously.

Lab Execution Card

[LAB A10] Inheritance (AddSub -> MulDiv).
[LAB A7] Interfaces and Abstract Classes.

Gemini Notebook

Behavioral Modifiers: Static, Final & Abstract

Static



Belongs to the class, not individual objects. Memory is allocated once and shared among all instances.

Final



Variables cannot be reassigned. Methods cannot be overridden. Classes cannot be inherited.

Abstract



Methods have no body. Classes cannot be instantiated and require a subclass to implement the details.

Lab Execution Card

[LAB A6] Counting the number of objects created using a static member function.

Gemini Notebook

Dynamic Data: Strings, Vectors & Wrappers

Strings

String

Immutable: Any modification creates a completely new string in memory.

StringBuffer

Mutable: Flexible length and content. Can be modified directly.

Vectors & Wrappers



Autoboxing: Automatic conversion of primitive data types into Objects.

Vector

Vector: A dynamic array that grows automatically but stores Objects Only. Primitive types must be boxed before insertion.

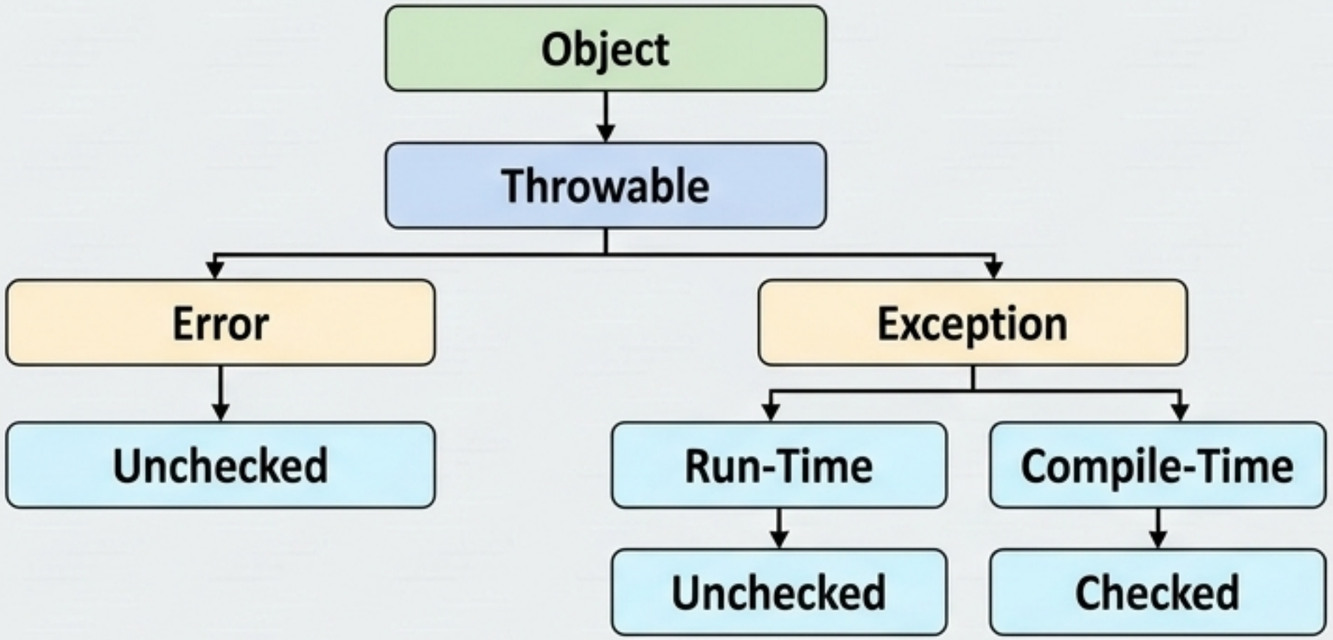


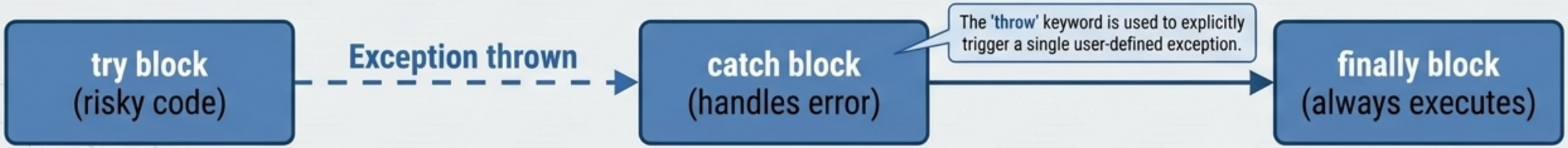
Lab Execution Card

[LAB A3] String/StringBuffer operations and Vector manipulation. [LAB B5] Reading integers using StringTokenizer.

Gemini Notebook

Safety Nets: Exception Handling Architecture





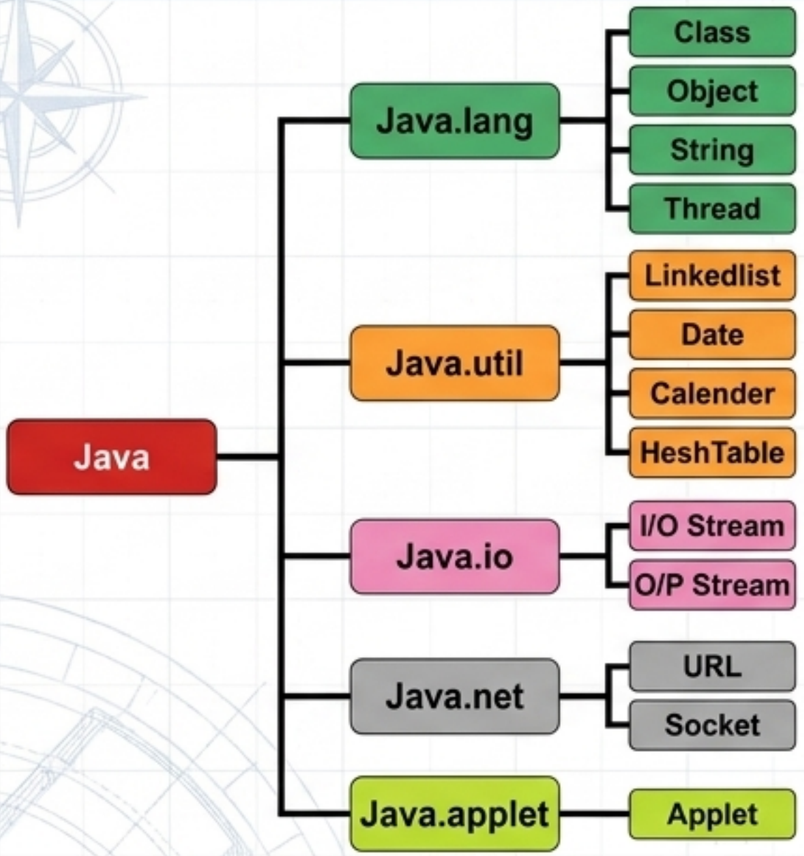
Lab Execution Card

[LAB A6] Exception-finally block.

Gemini Notebook

System Architecture: Packages & File I/O

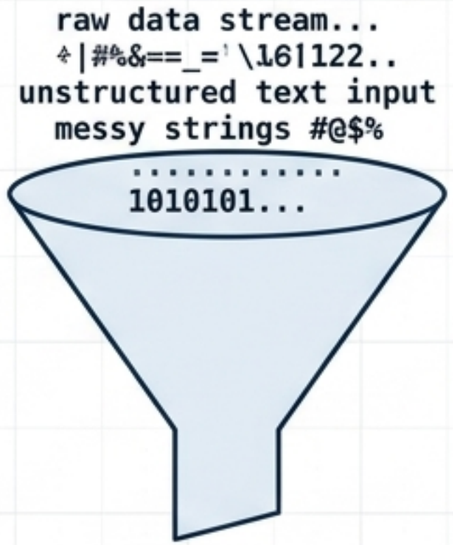
Packages



Packages act as containers to prevent naming conflicts and control access.

File I/O

File Reader / Scanner



Token 1
Integer: 42
String: "Hello"
Parsed Record
Data Block A

Tokens / Parsed Data

Lab Execution Card

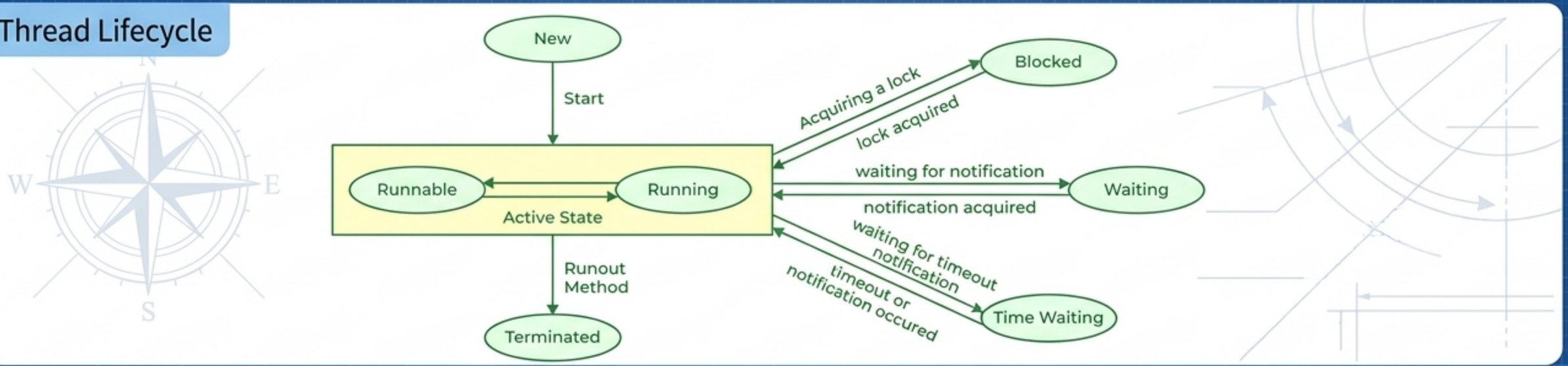
[LAB A8] Creating a user-defined package and importing it.
[LAB B6] Checking file properties (exists, readable, length in bytes).
[LAB B7] Analyzing text files to count characters, lines, and words.

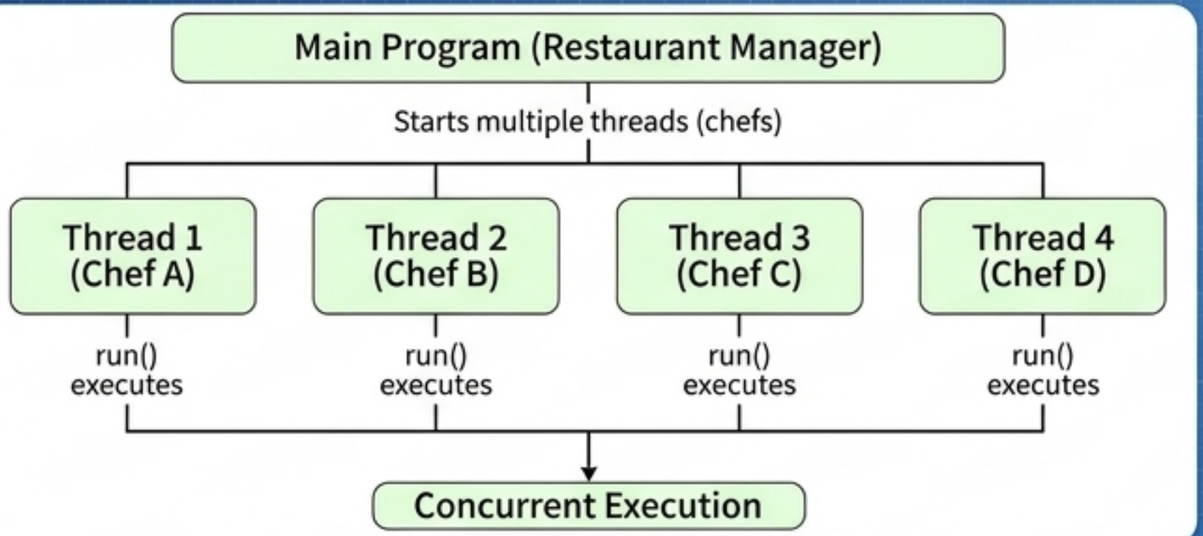
Gemini Notebook

Concurrency: Multithreading & Synchronization

Academic Blueprint

Thread Lifecycle





Theory Concept Card

Synchronization & Deadlock



Synchronization forces threads to take turns using shared resources. Deadlock occurs when two threads wait infinitely for locks held by each other.

Lab Execution Card

[LAB B10] Application displaying a deadlock between two threads locking shared Objects.

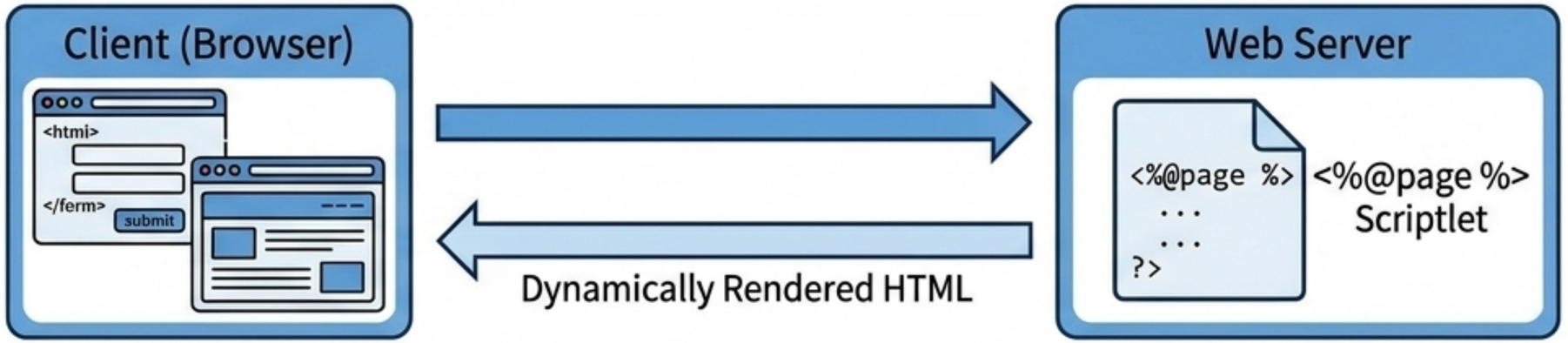
Gemini Notebook

Enterprise Extensions: JSP & JDBC

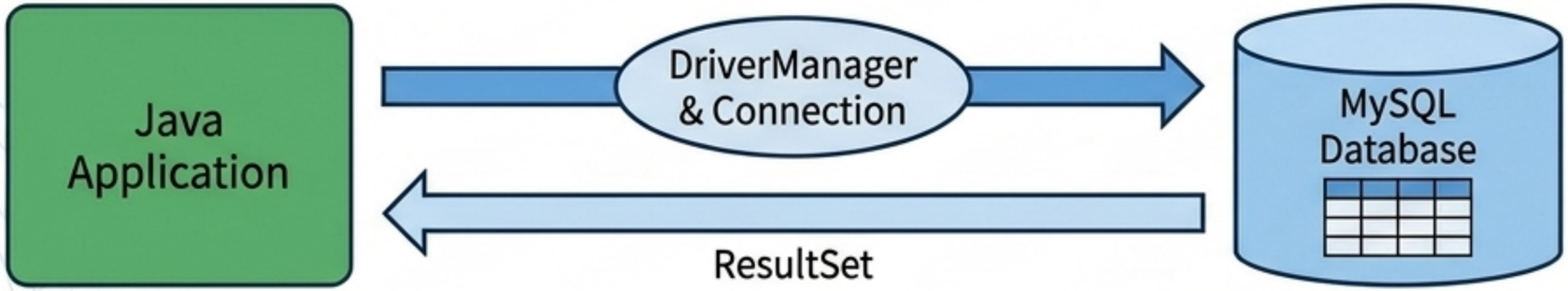
Academic Blueprint

Beyond desktop applications, Java powers dynamic web pages and database connections.

JSP - Java Server Pages



JDBC - Java Database Connectivity



Lab Execution Card

[LAB B8 & B9] JSP programs to select DB records and process numeric input.
[LAB B10 (**Part 2**)] Displaying data from an Employee DB table using JDBC.

Gemini Notebook

The Exam Recall Map

Academic Blueprint

Memory is spatial. Picture this blueprint during the exam to link theoretical questions to practical lab examples.

[NODE: FOUNDATIONS] (Types, Loops, Switch) — `[Labs A1, B1-B4]`

[NODE: OOP CORE] (Classes, Constructors, Modifiers) — `[Labs A4, A5, A6]`

[NODE: OOP PILLARS] (Inheritance, Interfaces, Overloading) — `[Labs A7, A9, A10]`

[NODE: DATA UTILITIES] (Strings, Vectors, Packages, Files) — `[Labs A2, A3, A8, B5-B7]`

[NODE: ADVANCED MECHANICS] (Exceptions, Threads, JSP, JDBC) — `[Labs B8, B9, B10]`

Gemini Notebook